# Translational Semantics - Rule-Based Model Transformation

Niels Van den Broeck, Robbe Stuer

## 1 Introduction

In this assignment, we will be translating our model into a Petri net using rule based model transformations.
First of all, there is a meta model for Petri nets given which we merged with our meta model to create a new meta model. Then, we translated our model into classes and associations from the Petri nets meta model, we kept all original classes and associations so that we could use them in additional rules if we were to expand on this.

## 2 Name-attributes

Before we head into the translation of our role-playing game, let's talk about how name-attributes are used in our rules to both speed up generation and to easily keep track of places/transitions/arcs. Names are assigned to all Petri net components (even when not used afterwards). This is done to avoid multiple general links between the meta model from assignment 5 and the Petri net model components. Later, we can check in the lhs or nac of a rule wether a place with a certain name exists. for example: condition = 'get_name(this) == f'hero_place_get_name(matched('tile1'))"'; with tile1:RAM_Tile. General links are still used to demonstrate their usage, and because they are cleaner in some cases.

## 3 Rules

### 3.1 Level and Inventory

For each level, we create a place. Later, we will use this place to link it with transitions between tiles (within this level) to restrict monsters from moving when the hero is not present in the level. This will be explained in detail later. Now that the Level place is made, it can be retrieved by simply checking if the place has name 'level_place_get_name(matched('level'))'.

The same is done for the inventory, where objectives are collected, and lives, which will decrease when stepping into a trap. (explained in detail later).

### 3.2 Tile

For every tile, we create a place for the hero to stand on, and a place for the monster to stand on. Only place per tile is needed for all the monsters, since there can only be 1 monster present per level.

### 3.3 Adjacency

For every pair of adjacent tiles, we create 2 transitions and 4 arcs for both the hero places and monster places (4 transitions and 8 arcs total). We do this so that the hero/monster can move between both tiles. (In our rule we only do this for tileX to tileY but of course since the association is symmetric, tileY to tileX will eventually be translated as well)

## 3.4 Collectables

The specification for collectables differs from previous assignment. We will no longer work with "collected_points", but we just keep track of the amount of objectives collected in general. Therefore, we separated the objectives and keys (no more inventory), by keeping a place for the objectives and a place for each key.

### 3.4.1 Objectives

For each objective, we create a place that represents the "uncollected" state. A transition is made from this place to the inventory. We want this transition to only fire when the hero is standing on a certain tile, so we add 2 arcs from the hero tile, one to and one from the transition. This will consume the marking, but place it right back after the transition fired.

### 3.4.2 Keys

The same is done for keys, only keys have a separate "collected" place, so that we can differ between collected keys when opening doors. The connection between the key and door is the same as the connection between the hero place and the pickup transition, we want the key place to have a marking (collected) so that the transition (use_door) can fire (see below for use_door rule).

## 3.5 Connect Doors

For every pair of connected doors, 2 transitions are made to simulate taking the door to the other level. Again like with adjacent tiles, this is done from door X to door Y so later with the reverse order as well.

## 3.6 Trap

Losing a life when stepping into a trap is not that complex. We simply find all transitions that go from a tile and go to a trap, we add an arc from our life_amount place we created earlier, to the transition. This way, each time this transition fires, a marking (life) is consumed from the live_amount place.

## 3.7 Monster Same Level Required

Since it is specified that the monsters can only move when it is present in the same level as the hero, We need a way to keep track of level the hero is currently in. This can be done by linking the level places with the use_door transitions. When the hero fires the transition use_door in level 1, the marking from this level gets consumed and generates a marking in the place of the level the door is connected to.

Now that we have knowledge of the hero's level, we can check if the monster may move by adding arcs from each adjacent_tile transition (for monsters, not hero) to its level place and back. Just like in use_door with a required key, and the presence of hero when picking up an item, for the monster to move, the marking in the level place should be present.

## 3.8 Place Hero and Monsters

Placing the hero and Monsters is easy. In the left hand side of the rule, we take the hero/monster and its standingTile, we get the petri net place of this tile and place a marking in this tile. An expansion we had to do for the hero is to put a marking in the level place where the hero is present as well.

## 3.9 Order of Actions

### 3.9.1 Actions

We want to have a turn based mechanism where we have 3 states. A state in which only the hero can move, a state in which the hero has the option to pick up the item that is present on his tile, and a state in which the monster can move. In order to achieve this, we made a place for each state and arcs so that we can cycle through them. As soon as the marking is in a certain state's place, these transitions can fire, otherwise it will be impossible. This is done by simply adding arcs from the state place to the transition that may fire, and an arc from the transition that may fire to the next state place. To prevent getting stuck when no transitions are possible, we made a transition from the collecting state to the monster state and a transition from the monster state to the hero state.

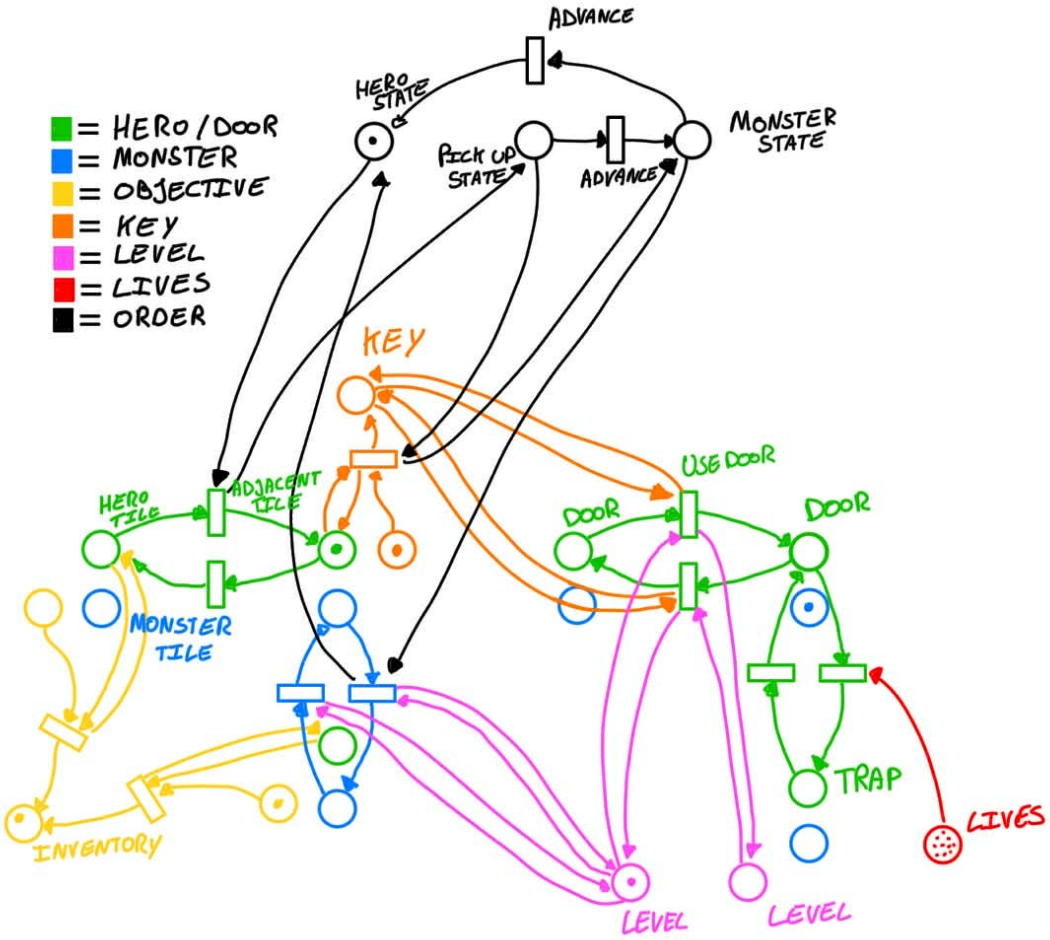



Figure 1: Drawing of all petri net concepts implemented in this assignment.